Making Final Project in Drafter

Kingsley Royal

now its time to make project for drafter

🌍 Travel Budget Planner — Building Your Drafter Website

We're ready to turn your project plan into an actual Python website using **Drafter**.

We'll stick closely to your original idea and the requirements for your CS1 final project. The first version will let users enter their travel information, calculate a daily spending budget, add planned expenses, and view a report.

1. Your website structure

Home Page — index

Welcome message and Start Planning button

Start Planning

Trip Input — start_planning

Destination, budget, flight, hotel, trip length, exchange rate

Calculate Budget

Budget Results — results_page

Remaining budget, category allocations, daily allowance

Add Expenses

Expenses Page

expenses_page

Enter planned purchases and their costs

View Report

Report Page

report_page

Total expenses and budget remaining

Your program will include **two data classes** (State and Expense) and use the 40/30/20/10 budget allocation rule.

2. Assignment requirements

Requirement	Our Implementation
3 if statements	Check trip length, budget, and planned expenses
1 loop	Add together expenses stored in a list
3 user inputs	Destination, budget, flight, hotel, days, currency rate, and expenses
5 pages	Home, Trip Input, Results, Expenses, Report
7 routes	Eight route functions
State data class	Stores trip data and expense objects
Second data class	Expense with name, category, and cost

I've checked the current Drafter documentation for its page, form, and route syntax.

3. Complete Python code for Drafter

Copy this into your Python project. The functions are grouped into sections so you can understand what each part does.

from drafter import *from dataclasses import dataclass, field# ----------# DATA CLASSES# ----------@dataclassclass Expense: name: str category: str cost: float@dataclassclass State: destination: str = "" total_budget: float = 0.0 flight_cost: float = 0.0 accommodation_cost: float = 0.0 length_of_stay: int = 1 exchange_rate: float = 1.0 currency_name: str = "" expenses: list[Expense] = field(default_factory=list)# ----------# HELPER FUNCTIONS# ----------def remaining_budget(state: State) -> float: return (state.total_budget - state.flight_cost - state.accommodation_cost)

4. Test your website

After running the program, use these sample inputs to check the calculations.

Sample trip inputs

Destination

Tokyo, Japan

Trip length

10 days

Budget

$3,000

Flight

$900

Accommodation

$800

Currency

JPY — rate 150

EXPECTED RESULTS

Remaining budget

$1,300

Daily spending

$117

Emergency savings

$130

Daily JPY (example rate)

¥17,550

Try adding a $25 museum ticket as a miscellaneous expense. The report should show $25 in planned additional expenses and $1,145 in uncommitted spending money.

5. What to do next

1. Paste the code into your Drafter Python editor.

2. Run the application and open the Home Page.
3. Enter the sample trip information above.
4. Test all five pages and their navigation buttons.
5. Add several planned expenses and check the report.

The version above is a first working draft, not a tested deployment in your specific Drafter environment. Drafter's documentation and examples are available at Drafter Documentation.

Once the basic functionality runs correctly, the next improvement should be making the website look more like your original mockup, with a better layout, colored headings, and a visual budget breakdown.

add necessary, concise comments

Here is your Travel Budget Planner code with **short, necessary comments**. The comments explain the important sections, calculations, conditions, and loop without making the code cluttered.

```
from drafter import *
from dataclasses import dataclass, field


# Stores information about each planned expense.
@dataclass
class Expense:
    name: str
    category: str
    cost: float


# Stores user inputs and expenses between pages.
```

```
@dataclass
class State:
    destination: str = ""
    total_budget: float = 0.0
    flight_cost: float = 0.0
    accommodation_cost: float = 0.0
    length_of_stay: int = 1
    exchange_rate: float = 1.0
    currency_name: str = ""
    expenses: list[Expense] = field(default_factory=list)



# Calculates money remaining after fixed trip costs.
def remaining_budget(state: State) -> float:
    return (state.total_budget - state.flight_cost
            - state.accommodation_cost)



# Calculates daily spending, keeping 10% for savings.
def daily_budget(state: State) -> float:
    spending = remaining_budget(state) * 0.90
    return spending / state.length_of_stay
```

These comments focus on **what each function does, why the conditions matter, how the loop works, and when** State **changes**. That's a good level of commenting for an introductory programming project without explaining every individual line.

change mischellaneous category to "Activities"

Sure! We'll replace **"Miscellaneous"** with **"Activities"** throughout your Travel Budget Planner while keeping the 40/30/20/10 budgeting rule.

You'll need to update three sections of your code.

1. Budget Calculations

In results_page, change:

miscellaneous = remaining * 0.20

To:

activities = remaining * 0.20

2. Budget Results Display

Change:

"Miscellaneous (20%): " + money(miscellaneous) + "\n",

To:

"Activities (20%): " + money(activities) + "\n",

3. Expense Category Dropdown

In expenses_page, change:

SelectBox("category", ["Food", "Transportation", "Miscellaneous"]),

To:

SelectBox("category", ["Food", "Transportation", "Activities"]),

Updated Budget Allocation

Category	**Percentage**
Food	40%
Transportation	30%
Activities	20%
Emergency Savings	10%
Total	100%

Everything else stays the same! Your expense tracking, daily budget calculation, data classes, and routes will continue working as before.

thoughts on asking if they want to do more eating, activities, or plan to travel a lot within their traveling

I think that's a great feature to add to your Travel Budget Planner, especially because it makes the budget personalized without making your CS1 project much more complicated.

Instead of giving every traveler the same 40/30/20/10 budget, your website could ask what they plan to prioritize during their trip and adjust the percentages automatically.

1. Add a Travel Preference Question

On your Trip Input Page, add:

What do you plan to spend the most money on during your trip?

Food — I want to try lots of restaurants and local cuisine.

Activities — I want to visit attractions and experience new things.

Transportation — I plan to travel around frequently.

Balanced — I want to divide my money evenly across my priorities.

Your suggested budget allocation

Food

40%

Transportation

30%

Activities

20%

Emergency Savings

10%

Choose a preference above to see how your budget changes.

2. How This Changes Your Python Project

You would add just one new attribute to your State data class:

travel_preference: str = "Balanced"

Then, in your results_page function, use conditional statements to adjust the percentages.

Adjusts budget categories based on travel preference.if state.travel_preference == "Food": food = remaining * 0.50 transportation = remaining * 0.20 activities = remaining * 0.20elif state.travel_preference == "Activities": food = remaining * 0.30 transportation = remaining * 0.20 activities = remaining * 0.40elif state.travel_preference == "Transportation": food = remaining * 0.30 transportation = remaining * 0.40 activities = remaining * 0.20else: food = remaining * 0.40 transportation = remaining * 0.30 activities = remaining * 0.20# Emergency savings always stays at 10%.savings = remaining * 0.10

You would also add a selection input to start_planning and update calculate_budget to receive and save the user's choice.

3. Why I Recommend This

This feature would make your application better in several ways:

- More personalized: A traveler interested in food gets a different budget from someone who wants to explore multiple cities.
- Uses meaningful conditionals: The if, elif, and else statements directly affect the budget calculations.
- Keeps calculations simple: You're only changing percentages, so you don't need external APIs or complicated libraries.
- Fits your existing pages: You can add it to your Trip Input Page without needing a sixth page.

One important point: These percentages are suggested starting points, not actual estimates of how expensive food or transportation will be in a particular country.

My recommendation: Add this feature and call it Travel Spending Preference. It's one of the easiest ways to make your final project feel more interactive and personalized while keeping it manageable for CS1.

update code

Updated Travel Budget Planner Code

I've updated your Drafter website to include the Travel Spending Preference feature, with the changes you requested:

- Added a dropdown asking whether the user prioritizes Food, Activities, Transportation, or Balanced spending.
- Added travel_preference: str to your State data class.
- Added if, elif, and else statements to customize budget percentages.
- Replaced all references to "Miscellaneous" with "Activities".
- Kept the five pages, expense tracking, currency conversion, and concise comments.

The code uses Drafter's documented SelectBox and form parameter conventions.

Drafter Documentation+1

Complete Updated Python Code

Replace your previous code with this version.

from drafter import *from dataclasses import dataclass, field# Stores each planned travel expense.@dataclassclass Expense: name: str   category: str   cost: float# Stores trip information between pages.@dataclassclass State: destination: str = ""   total_budget: float = 0.0   flight_cost: float = 0.0   accommodation_cost: float = 0.0   length_of_stay: int = 1   exchange_rate: float = 1.0   currency_name: str = ""   travel_preference: str = "Balanced"   expenses: list[Expense] = field(default_factory=list)# Calculates money left after fixed expenses.def remaining_budget(state: State) -> float: return (state.total_budget - state.flight_cost   - state.accommodation_cost)# Calculates daily spending with 10% saved.def daily_budget(state: State) -> float: spending = remaining_budget(state) * 0.90

How the New Feature Works

Here's an example of how your budget changes based on the traveler's preference.

Travel preference

Example remaining budget: $1,300

Balanced

Food

40%

$520.00

Transportation

30%

$390.00

Activities

20%

$260.00

Emergency Savings

10%

$130.00

Daily spending allowance

$117.00

Based on a 10-day trip. Changing preferences redistributes category budgets without changing the total daily allowance.

What Changed in Your Code

Section	**Update**
State	Added travel_preference: str
start_planning	Added travel preference dropdown
calculate_budget	Saves the selected preference
budget_categories	New helper function using if/elif/else
results_page	Displays personalized spending categories and daily amounts
expenses_page	Uses Activities instead of Miscellaneous
report_page	Displays the personalized category budgets

Your project still meets the assignment requirements of at least 5 pages, 7 routes, 3 conditionals, 1 loop, 3 inputs, and 2 data classes.

One thing to keep in mind: the report treats planned expenses as part of the category budgets, not additional costs on top of those budgets. Your emergency savings stays at 10%, regardless of which preference the user chooses.

The next useful improvement would be to show how much money remains in each category after planned expenses, rather than only showing the overall uncommitted balance.

making the authorship to my code. customize the following to accomidate. add after import statemetns set_website_title("Corgi Cafe")
set_site_information(
author="Your Name",
description="Order snacks for very good dogs.",
sources="Thanks to the Drafter docs and my study group.",
planning=["plan.pdf", "design.pdf"],
links=["https://github.com/your-username/your-repository"]
)
hide_debug_information()
set_website_framed(False)

Add the following code directly after your import statements in your Travel Budget Planner project.

what do these errors mean on github

These are **GitHub Actions warnings and notices**, not necessarily errors in your Travel Budget Planner's Python code.

Your screenshot shows **3 warnings and 2 notices** related to how GitHub builds and deploys your website.

The good news is that none of these messages directly indicates that your Python code is broken.

1. What Each Warning Means

Message	**Meaning**	**Severity**
Node.js 20 is deprecated	Your GitHub Actions workflow uses an action that targets an older Node.js version.	Warning
No file matched requirements*.txt	GitHub searched for a Python dependency file but couldn't find one matching that pattern.	Warning

Node.js 20 warning during deploy	The deployment action also targets Node.js 20.	Warning
Ubuntu runner migration	GitHub is notifying you about changes to the operating system used to build your project.	Notice
Ubuntu runner migration during deploy	The same notice appears during deployment.	Notice

2. The Most Relevant Warning for Your Project

The warning I'd pay attention to first is:

No file matched … requirements*.txt

A requirements.txt file lists Python libraries that your project needs.

For your Travel Budget Planner, that includes **Drafter**.

You can create a file named requirements.txt in your GitHub repository containing:

drafter

Your project uses dataclasses, but that module is included with modern Python, so you don't need to list it separately.

However, the warning may simply come from an optional dependency-caching step. A missing requirements.txt file does not automatically mean your website failed to build.

3. What About the Node.js Warnings?

GitHub is warning that these actions use an older Node.js version:

- astral-sh/setup-uv@v6
- actions/deploy-pages@v4

GitHub is automatically running them using Node.js 24. This is a compatibility warning about the deployment tools, **not your Python program**.



GitHub+1

You don't need to rewrite your Travel Budget Planner to address it. Updating the GitHub Actions workflow may resolve the warnings later.

4. The Ubuntu Notices

GitHub is also announcing changes to the Ubuntu operating system used by its runners.

These are informational notices. No change to your Python code is required.

- **must use the Drafter python library to build the website**
- **The website must meet the requirements linked at the top of this assignment. Missing two or more requirements means you might fail this assignment entirely. The same is true if your website doesn't work.**

Worked for 9s

I'll compare the project we designed against the requirements you listed, then check the GitHub repository to see whether the deployed code can be verified. The key distinction is between meeting the feature checklist and having a website that actually runs.

Searched the web

Your assignment is emphasizing two things: your website must use Drafter, and it must actually work. Meeting the programming requirements alone isn't enough.

I checked your Travel Budget Planner GitHub repository and found that your GitHub Actions deployment completed successfully on November 12, 2026.

GitHub

That's encouraging, but a successful deployment does not guarantee that every button and page works.

Your Project Requirements Check

Based on the code currently in your GitHub repository:

Requirement	Your Project	Status
Uses Drafter Python library	from drafter import *	Present
At least 3 if statements	Budget validation, preferences, warnings	Present
At least 1 loop	Loops through planned expenses	Present

At least 3 user inputs	Destination, budget, flight, hotel, days, etc.	Present
At least 5 pages	Home, Input, Results, Expenses, Report	Present
At least 7 routes	8 decorated route functions	Present
State data class with 4+ attributes	9 attributes	Present
Second data class	Expense	Present
List containing second data class	expenses: list[Expense]	Present
Website works correctly	Must test published website	Not verified

The code appears to include all the programming elements you've told me about.

GitHub

Skip To Content

Dashboard

-
-
-
-
-
-
-
- **svg**
-

1. **26F-CISC106**
2. **Pages**
3. **Final Project Requirements (Specs)**

2026 Fall

- Home
- Grades71
- Modules
- BlockPy Dashboard
- Syllabus
- Announcements
- UD Tutoring

Final Project Requirements (Specs)

Final Project Roadmap

The final project will involve creating a web application with Drafter. The different steps of the project will be spread across the last several weeks of class.

- **Week 10: Brainstorming & Ideation - You will come up with an idea for your web application. You will brainstorm its goal, details, and functionality.**
- **Week 11: Design - You will plan how the site will look and how you will implement the requirements listed on this page.**
- **Week 12: Development & Deployment - You will develop the code for your website and deploy it.**
- **Week 13: User Testing - You will show your website to peers and get feedback on their experiences using it.**
- **Week 14: Updates based on Feedback - You will make updates based on the feedback you receive and reflect on the process of creating your web application.**

Site Functionality Requirements

Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

- **At least 7 routes.**
- **At least 5 Pages, or 10 FragmentsLinks to an external site., or a combination thereof.**
- **The implementation of the *State* dataclass must have at least 4 fields.**

    - **At least one field must be a list of class instances or a dictionary of class instances.**
        - **This means that there must be a second dataclass.**
    - **At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.**
- **At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.**
- **At least 3 meaningful if statements.**
- **At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.**
- **The site should have a legitimate purpose or functionality.**
- **No global variables (global constantsLinks to an external site. are fine).**
- **You must use the Drafter library.**
- **Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.**

More creative and impressive designs are given flexibility on the above requirements - impress us! However, impressive websites tend to do more than the above requirements. Further, if you miss more than 2 requirements, you will get no credit for that submission. For example, missing a second dataclass means you would be missing not only that requirement, but also the list or dictionary of such dataclass instances, and the loop iterating over that list. This would mean that the submission would get a zero.

Code Requirements

Make sure your program and website runs; broken programs do not earn points. This means that if your program contains a syntax error or runs and doesn't deploy a website, you get zero points for this assignment, and are subject to any other penalties listed in the syllabus.

Once you are finished with your project code, check over the following:

- **Chosen good variable names (check last two sections of Chapter 1B1Links to an external site.)**

- Unit tested your functions where possible, being valid and thoroughLinks to an external site.: each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).
    - If a function can't be unit tested, you must note as much in a docstring or comment.
- Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)
- Included types in your function headers
- Included good docstrings Links to an external site.as the first line of your function's body
- Decomposed complicated functions with helper functions

Your TA will be grading your code's quality based on the ideas above.

Repository Requirements

Your final project's GitHub repository should meet the following requirements.

- Planning document should be uploaded to the repository.
- Website should be deployed without error.
- Site information should be set in main.py using set_site_information(...).
- When changing/adding files in the repository, commit messages should be concise and descriptive, not the default "Update (filename)" message.
- A video fulfilling the following requirements should be linked in the repository.
    - Shows functioning deployed website
    - Audible and visible
    - Accesses and explains all necessary parts of the website
- All conversations with any AI should be uploaded to the repository.

The above are either present on the final rubric or are necessary for your site to be graded.

Travel Budget Planner — Full Final Project Requirements Review

Now that you've shared the official CISC106 Final Project Requirements, we can check your Travel Budget Planner against the actual grading criteria.

Your project already has most of the required functionality. However, there are several code-quality and submission requirements we still need to address before your project is ready for grading.

1. Site Functionality Requirements

Based on the latest code we've developed together:

Requirement	Your Travel Budget Planner	Status
At least 7 routes	8 @route functions	Met in code
At least 5 pages	Home, Input, Results, Expenses, Report	Met in code
State dataclass with 4+ fields	9 fields	Met
Second dataclass	Expense with 3 attributes	Met
List of dataclass instances	expenses: list[Expense]	Met
4 State fields meaningfully modified in routes	Trip inputs and expense list	Met
At least 3 input fields	9+ input fields	Met
At least 3 meaningful if statements	Validation, spending preferences, warnings	Met
At least 1 meaningful loop	Iterates through expenses and adds their costs	Met
Legitimate purpose	Personalized travel budgeting	Met
No global variables	No mutable global variables in our planned code	Met
Uses Drafter	from drafter import *	Met

Advanced techniques documented	Some explanations, but documentation needs improvement	Improve

This is an assessment of the code shared in our conversation, not a completed test of your current GitHub version or deployed website.

2. Code Quality Requirements — Important Changes

Your professor specifically requires docstrings, tests, type annotations, and manageable code.

Requirement	Current Situation	Action
Good variable names	Names like total_budget, flight_cost, and daily_budget are clear	Keep
Unit tests for each route	We haven't written these yet	Add tests
Lines under 120 characters	Most of our code is within this limit	Verify
Function type annotations	Our functions already include parameter and return types	Keep
Docstrings in functions	We have comments, not actual docstrings	Add docstrings
Helper functions	Already use remaining_budget, daily_budget, and budget_categories	Keep and improve
Document advanced techniques	field(default_factory=list) and some typing could use explanations	Add documentation

Important: Comments and docstrings are different

Your professor specifically asks for docstrings as the first statement inside each function.

Currently, your code uses comments like:

Calculates daily spending with 10% saved.def daily_budget(state: State) -> float: spending = remaining_budget(state) * 0.90 return spending / state.length_of_stay

You should change this to:

def daily_budget(state: State) -> float: """Calculate daily spending while reserving 10% for emergencies.""" spending = remaining_budget(state) * 0.90 return spending / state.length_of_stay

Every function in your project should have a meaningful docstring, including your page and route functions.

You also need unit tests

Your professor says each route should have at least one unit test when possible.

For example, here's a simple test for your remaining_budget helper function:

def test_remaining_budget(): """Test the remaining budget after fixed expenses.""" state = State(total_budget=3000, flight_cost=900, accommodation_cost=800) assert remaining_budget(state) == 1300

This checks that your calculation produces the correct result.

You'll also need tests for your route functions, especially calculate_budget and save_expense, including cases where inputs are invalid. These tests should check both the returned page and relevant changes to State.

3. Repository Requirements

These are separate from the website's programming requirements.

GitHub submission checklist

4/6 completed

Upload your planning document

Include your brainstorming and website design documents in the GitHub repository.

Verify that the deployed website works

Check all five pages, form inputs, buttons, calculations, and expense saving.

Include set_site_information(...)

Your authorship settings are already prepared; confirm they are in main.py.

Write descriptive commit messages

Use messages such as "Add spending preference selection" instead of "Update main.py".

Record and link a demonstration video

Show the deployed website working, with audible and visible explanations of its features.

Upload all AI conversations

Include the conversations you used while planning, developing, and debugging the website.

4. What I Recommend Fixing First

I would prioritize the following work in this order:

1. Confirm the website runs properly. Test whether all pages and buttons work, especially the forms that pass values between routes.
2. Add docstrings to every function.
3. Write unit tests for your helper functions and routes.
4. Document any advanced techniques, including how the two dataclasses work together.
5. Finish the repository requirements, especially planning files, AI transcripts, and the demonstration video.

This isn't a guarantee of satisfying any additional requirements in the linked rubric.

The most serious issue would be a program that doesn't run or deploy. Your rubric explicitly says broken programs receive zero points, so that takes priority over visual improvements.

5. My Assessment

Your planned application satisfies the main feature checklist.

Code testing, documentation, and submission evidence still need attention.

I recommend that our next step be to update your existing Travel Budget Planner Python code with proper docstrings and meaningful unit tests, while preserving the five pages, eight routes, personalized spending preferences, currency conversion, and Activities category.

That will address the most important remaining code-quality requirements without unnecessarily complicating your CS1 project.

update code to meet requirementd

Updated Travel Budget Planner — Rubric-Ready Version

I've revised your project to address the official CISC106 requirements while keeping it beginner-friendly.

This version includes the following improvements:

- 5 main pages and 8+ routes
- 2 dataclasses: State and Expense
- 9 State attributes, including a list of Expense objects
- 3+ meaningful conditionals and a loop through saved expenses
- Docstrings at the beginning of every function
- Type annotations for function parameters and return values
- Unit tests for helper functions and every route
- Personalized spending preferences, Activities spending, and currency conversion
- Website authorship and information settings

I also checked Drafter's current documentation. Drafter supports testing route functions directly with its built-in assertions, which means you don't need to install a separate testing library.

Drafter Documentation+1

One important change: I've removed extra navigation buttons from pages containing input forms. Drafter submits the form fields when a button or internal link is clicked, so navigation from those pages needs careful handling.

Drafter Documentation+1

Complete updated main.py

Replace your existing main.py with the following code.

from drafter import *from dataclasses import dataclass, field# Website title and authorship information.set_website_title("Travel Budget Planner")set_site_information(author="Kingsley Royal", description="Plan travel expenses, calculate daily budgets, " "and organize spending based on travel preferences.", sources="Drafter documentation, ChatGPT assistance, " "and CISC106 course materials.", planning=[], links=["https://github.com/royalk-ai/travel-budget-planner"])set_website_framed(False)# --

----------# DATA CLASSES# ----------@dataclassclass Expense: """Represent one planned expense during a trip.""" name: str category: str cost: float@dataclassclass State: """Store trip details and planned expenses between pages."""

2. What This Code Now Meets

Requirement	Implementation
7+ routes	8 @route functions
5+ pages	Home, Input, Results, Expenses, Report, plus confirmation/error pages
State dataclass	9 attributes
4 meaningfully modified State fields	calculate_budget updates 8 fields; save_expense updates the list
Second dataclass	Expense with name, category, cost
List of dataclass instances	expenses: list[Expense]
3+ inputs	7 trip inputs plus travel preference and expense inputs
3+ if statements	Input validation, preferences, spending warnings, expense checks
Meaningful loop	Totals planned expenses and displays each one
Docstrings	Included in every function
Function types	Parameters and return types included
Unit tests	Each route has at least one test
Website authorship	set_site_information(...)
No mutable global variables	Application data stays in State

3. Important Notes Before Deployment

First, add your planning files. I set planning=[] because I cannot confirm the filenames of your uploaded planning documents. Once you have added them to your repository, update that field with their real filenames.

Second, keep your tests visible during development. I deliberately left out hide_debug_information() so you can inspect Drafter's test results. After testing, you can choose to restore it.

Third, test the deployed website manually. The code includes automated assertions, but I haven't executed this exact program in your GitHub/Pyodide environment. Check that the input form, route navigation, calculations, expense saving, and reports all work after publishing.

4. Suggested Test Scenario

Try this in your deployed website

Destination

Tokyo, Japan

Trip length

10 days

Total budget

$3,000

Flight

$900

Accommodation

$800

Preference

Activities

EXPECTED RESULTS

Remaining budget

$1,300

Daily allowance

$117

Food	$390

Transportation	$260
Activities	$520
Emergency savings	$130

Then add a $25 museum ticket under Activities. The expense report should show $25 in planned expenses and $1,145 in uncommitted spending funds.

Your next priority is to replace main.py, run the tests, and check the actual website. If Drafter reports a failure, the error message and the relevant test result will tell us which part needs adjusting.

how would you describe the new changes made in list format

Summary of Changes Made to Travel Budget Planner

1. **Added Function Docstrings:** Added short descriptions to every function explaining its purpose and functionality.

2. **Added Unit Tests:** Created tests to check budget calculations, page navigation, user inputs, and expense tracking.

3. **Improved Input Validation:** Added conditional statements to prevent invalid budgets, negative expenses, missing destinations, and incorrect trip lengths.

4. **Improved Budget Calculations:** Organized calculations into helper functions to make the code easier to understand and maintain.

5. **Updated Travel Spending Preferences:** Allowed users to prioritize Food, Transportation, Activities, or Balanced spending, automatically adjusting their budget percentages.

6. **Improved Expense Tracking:** Used the Expense dataclass and a loop to store planned expenses and calculate their total cost.

7. **Updated State Management:** Ensured travel information and planned expenses are stored and updated using the State dataclass.

8. **Improved Code Organization:** Added clear sections, concise comments, descriptive variable names, and type annotations.

9. **Updated Website Information:** Added the website title, author information, project description, sources, and GitHub repository link.

10. **Improved Navigation:** Updated page buttons and routes to make navigating between the five main pages easier.

11. **Added Budget Warnings:** Displayed messages when a user's daily spending allowance is low or their planned expenses exceed the available budget.

12. **Aligned Code With Project Requirements:** Updated the program to address the CISC106 requirements for routes, pages, dataclasses, inputs, conditionals, loops, docstrings, and unit tests.

These describe the changes made to the code; the updated website still needs to be tested to confirm everything works correctly.